

Web Science
Assignment – 7

Institute of Web Science and Technologies
Universität Koblenz-Landau

Group name: FOWLER

Name: Jeevan Saraf
jeevansaraf@uni-koblenz.de
Matriculation no: **222100445**



Name: Parth Chavda
parthchavda@uni-koblenz.de
Matriculation no: **222100495**

Name: Prajna Shetty
prajnashetty73@uni-koblenz.de
Matriculation no: **222100479**

Name: Ritika Bhandari
rbhandari@uni-koblenz.de
Matriculation no: **222100563**

1 Generative Models for the Web (15 points)



1 * Create a probabilistic generative model of text by sampling from the distribution of 1) word lengths 2) frequency of each character in the given text, similar to the one presented in the video slides1

Modelling choices:

- Your model should generate one word at a time according to the distribution of the word lengths.
- Within each word, generate characters according to the distribution of character frequencies.
- Your model should only consider lowercase letters [a~z] and numbers [0~9].

All uppercase letters should be converted to lowercase. Other characters such as punctuations should be excluded.

Generate a text of 5, 000 words with your model. Please upload your generated text as an individual file, but do not include it in the PDF document.

2. * Plot the probability distribution of word lengths of 1) the original text 2) the text you generated in one plot.

Answer:

```
import regex as re
import random as rand
import matplotlib.pyplot as plt

def read_file():
    with open("/Users/jeevan/Downloads/files/simple-english-wikipedia.txt", "r") as file:
        text = file.read()
    return text

def word_length_distribution(clean_text):

    word_length_distribution = {}
    word_length_freq_prob = {}

    total_words = len(clean_text)

    for word in clean_text:

        word_length = len(word)

        if word_length not in word_length_distribution:
            word_length_distribution[word_length] = 0
            word_length_distribution[word_length] += 1

    sum_of_prob = 0

    for word_len, word_count in word_length_distribution.items():
```

```
    abc = word_count / total_words
    word_length_distribution[word_len] = abc
    sum_of_prob += abc
    word_length_freq_prob[word_len] = sum_of_prob

word_length_distribution = dict(sorted(word_length_distribution.items()))

return word_length_distribution, word_length_freq_prob

def character_distribution(clean_text):
    character_frequency = {}
    character_frequency_probability = {}

    for word in clean_text:
        for c in word:
            if c in character_frequency:
                character_frequency[c] += 1
            else:
                character_frequency[c] = 1

    total = sum(character_frequency.values())

    sum_of_prob = 0
    for word_len in character_frequency.keys():
        sum_of_prob += (character_frequency[word_len] / total)
        character_frequency_probability[word_len] = sum_of_prob
    return character_frequency_probability

def random_word_length(word_len_dist, random_num):
    for word_length in word_len_dist.keys():
        if random_num < word_len_dist[word_length]:
            return word_length

def random_characters(char_dist, random_num):
    for char in char_dist.keys():
        if random_num < char_dist[char]:
            return char

def generate_text(word_len_distribution, char_distribution):
    text_words = []
    for i in range(0, 5000):
        random_num = rand.random()
        word_length = random_word_length(word_len_distribution, random_num)

        word = ""

        for j in range(0, word_length):
            random_num = rand.random()
```

```
        char = random_characters(char_distribution, random_num)
        word += char
        text_words.append(word)
    return text_words

def plot_graph(org_text_dist, gen_text_dist):
    plt.plot(org_text_dist.keys(),org_text_dist.values(), label="Original Text")
    plt.plot(gen_text_dist.keys(),gen_text_dist.values(),label="Generated Text")
    plt.legend()
    plt.xlabel('Word length')
    plt.ylabel('Probability distribution')

if __name__ == "__main__":

    text = read_file()
    text = text.lower().replace(',',' ')
    clean_text = re.findall(r'[a-z]*[0-9]*',text)
    while("" in clean_text):
        clean_text.remove("")

    #print(clean_text)

    word_length_dist, word_length_freq_probab = word_length_distribution(clean_text)
    #print(word_length_dist)

    character_dist = character_distribution(clean_text)
    #print(character_dist)

    num = 5000
    generated_text = generate_text(word_length_freq_probab, character_dist)

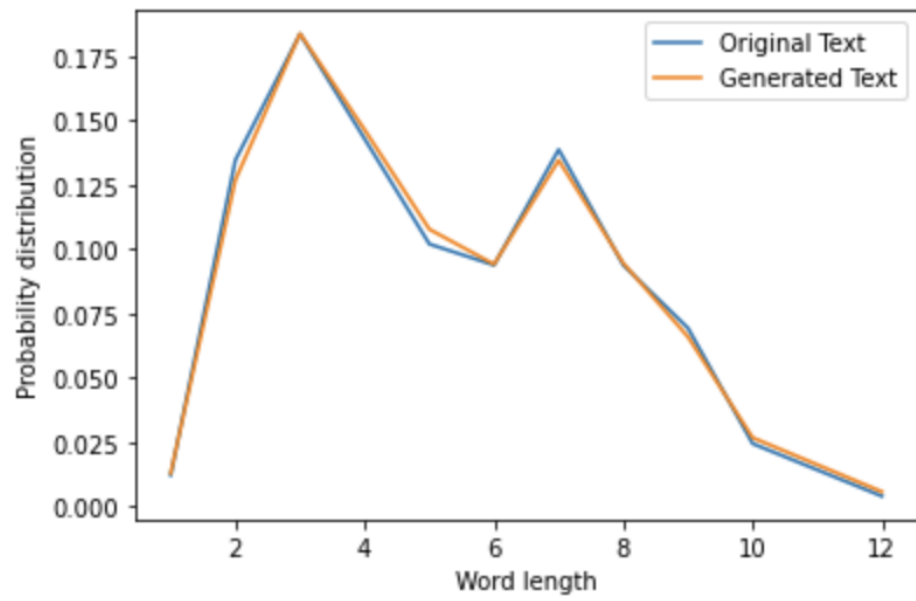
    with open("output_assignment7WS.txt", "w") as f:
        for word in generated_text:
            f.write(word+" ")

    gentext_dist, gen_freq_dist = word_length_distribution(generated_text)
    #print("gentext_dist", gentext_dist)

    while("" in gentext_dist):
        gentext_dist.remove("")

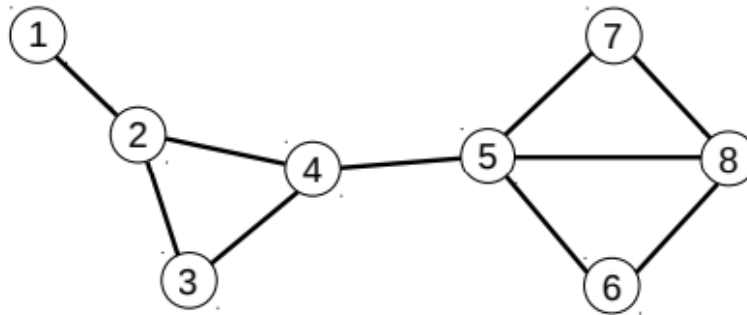
    plot_graph(word_length_dist, gentext_dist)
```

Output:



2 Modeling the Web as a Graph (15 Points)

Consider the following undirected graph G:



- 1 Write down the adjacency matrix A of the graph G.
2. The diameter of a graph is defined as the longest shortest path in the graph. The shortest path between two vertices u and v is defined as the distance between u and v: $\text{dist}(u, v)$.
Write the definition of the diameter as a mathematical formula, and give the diameter d of the graph G.
3. Usually we do not store the adjacency matrix directly since it is too large. Instead, we use one line: “u v” to represent an undirected edge between vertices u and v.
* Write a function to read the file graph.dat and convert it to the adjacency matrix.
4. * Write a function to compute the Gini coefficient g of the degrees of vertices in G. The function should have the adjacency matrix of G as its only parameter.
5. A triangle in an undirected graph $G = (V, E)$ is defined as three vertices that are adjacent to one another.
Write the definition as a mathematical formula.
* Write a function to compute the number of triangles t in the graph G. The function should have the adjacency matrix of G as its only parameter.

ANSWER:

1. Adjacency Matrix:

```
[0, 1, 0, 0, 0, 0, 0, 0]
[1, 0, 1, 1, 0, 0, 0, 0]
[0, 1, 0, 1, 0, 0, 0, 0]
[0, 1, 1, 0, 1, 0, 0, 0]
[0, 0, 0, 1, 0, 1, 1, 1]
[0, 0, 0, 0, 1, 0, 0, 1]
[0, 0, 0, 0, 1, 0, 0, 1]
[0, 0, 0, 0, 1, 1, 1, 0]
```
2. The definition of the diameter as a mathematical formula, and give the diameter d of the graph G.
 - a. Diameter formula: The diameter of a graph is defined as the largest shortest path distance in the graph.
 $\text{diameter}(G) = \max(\text{d}(x, y))$
 where d is the minimum distance between any two nodes x & y

b. Diameter of G: 4

3. A function to read the file graph.dat and convert it to the adjacency matrix:

```
import pandas as pd
import csv
import numpy
```

```
def plot():
```

```
    data = pd.read_csv('graph.dat', sep="\t")[1:]
    df = pd.DataFrame(data["% sym unweighted"].apply(lambda x: pd.Series(str(x).split("
")))).apply(pd.to_numeric)
    graphDat = df.to_numpy()

    maxVal = graphDat.max()+1
    adjacencyMatrix = numpy.zeros((maxVal, maxVal))

    adjacencyMatrix[graphDat[:,0], graphDat[:,1]] = 1
    adjacencyMatrix[graphDat[:,1], graphDat[:,0]] = 1

    return adjacencyMatrix[1:,1:]
```

```
matrix = plot()
matrix
```

Output:

```
Out[1]: array([[0., 1., 0., 0., 0., 0., 0., 0.],
               [1., 0., 1., 1., 0., 0., 0., 0.],
               [0., 1., 0., 1., 0., 0., 0., 0.],
               [0., 1., 1., 0., 1., 0., 0., 0.],
               [0., 0., 0., 1., 0., 1., 1., 1.],
               [0., 0., 0., 0., 1., 0., 0., 1.],
               [0., 0., 0., 0., 1., 0., 0., 1.],
               [0., 0., 0., 0., 1., 1., 1., 0.]])
```

4. A function to compute the Gini coefficient g of the degrees of vertices in G . The function should have the adjacency matrix of G as its only parameter.

```
def gini(x):
    matrix = numpy.add.reduce(x)
    x = 0
    for i, j in enumerate(matrix[:-1], 1):
        x += numpy.sum(numpy.abs(j - matrix[i:]))
    giniCoef = x / (len(matrix)**2 * numpy.mean(matrix))

    return giniCoef
```

```
coef = gini(matrix)
coef
```

Output: _____

Out[5]: 0.1875

5. A triangle in an undirected graph $G = (V, E)$ is defined as three vertices that are adjacent to one another.

- a. The definition as a mathematical formula:

For every edge $e = (u, v)$ in Edge set E , where u, v belong to Vertex set V .

If the adjacency lists of u and v have a common vertex w , where w belongs to V .

Then (u, v, w) is a triangle.

- b. Write a function to compute the number of triangles t in the graph G . The function should have the adjacency matrix of G as its only parameter.

```
def multiply(A, B, l):
    a = [[None] * l for i in range(l)]

    for i in range(l):
        for j in range(l):
            a[i][j] = 0

    for i in range(l):
        for j in range(l):
            a[i][j] = 0
            for k in range(l):
                a[i][j] += A[i][k] * B[k][j]
    return a

def triangles(matrix):
    l = len(matrix)
    c = multiply(matrix, matrix, l)
    d = multiply(matrix, c, l)

    num = 0
    for i in range(l):
        num += d[i][i]
    return num // 6

t = triangles(matrix)
t
```

Output: _____

Out[26]: 3.0